

Лабораторна робота №4

Моделювання режиму очікування в цифрових пристроях за допомогою оператора **wait**

Мета роботи: отримати практичні навички застосування оператора **wait** при моделюванні різних режимів очікування у Active-HDL. Навчитися виконувати порівняння часових діаграм.

Теоретична частина

Присвоєння нових значень сигналам в процесах відбувається тільки після припинення процесу. Для задання умов припинення процесу використовується оператор **wait**. Якщо процес містить оператор **wait**, він виконує послідовно всі оператори до оператора **wait**. Після цього процес припиняється і очікує виконання умови, заданої в операторі **wait**. Як тільки це відбувається, процес поновлюється і виконує всі оператори, поки знову не зустрине оператор **wait**.

Існує три типи оператора **wait**:

- **wait for** *time_expression* - очікування на протязі певного часу;
- **wait until** *condition* - очікування виконання логічного виразу;
- **wait on** *sensitivity_list* - очікування зміни значення сигналу із списку:

Якщо змішуються декілька таких умов, вони утворюють четверту конструкцію, яка називається складною умовою.

Тип оператора wait	Опис	Приклади
wait for <i>time_expression</i>	Припиняє процес на визначений час; Час може бути заданий безпосередньо, або як вираз, результатом якого є значення часу. Використовуються в основному для моделювання на випробувальних стендах.	wait for 10 ns; wait for ClkPeriod / 2;
wait until <i>condition</i>	Припиняє процес доти, доки задана умова не стане істинною завдяки зміні будь-якого сигналу, що містяться в умові. При цьому, якщо сигнал не змінюється, wait until не поновить процес, навіть якщо сигнал задовільняє умову.	wait until CLK='1'; wait until CE and (not RST); wait until IntData > 16;
wait on <i>sensitivity_list</i>	Припиняє процес доти, доки не відбудеться подія з будь-яким з сигналів, вказаних в списку чутливості. Іншими словами, процес поновлюється тільки тоді, коли зміниться значення хоча б одного сигналу з списку чутливості.	wait on CLK; wait on Enable, Data;
складний wait	Містить комбінацію двох або трьох різних форм оператора wait .	wait on Data until CLK='1'; wait until Clk='1' for 10 ns;

Якщо симулятор знаходить оператор **wait** відразу на початку процесу, то процес буде негайно припинений і жодний оператор не буде виконаний. Якщо ж оператор **wait** розташований в кінці процесу, спочатку будуть виконані всі оператори, що розташовані перед оператором **wait**, після чого процес буде припинений.

Порядок виконання роботи:

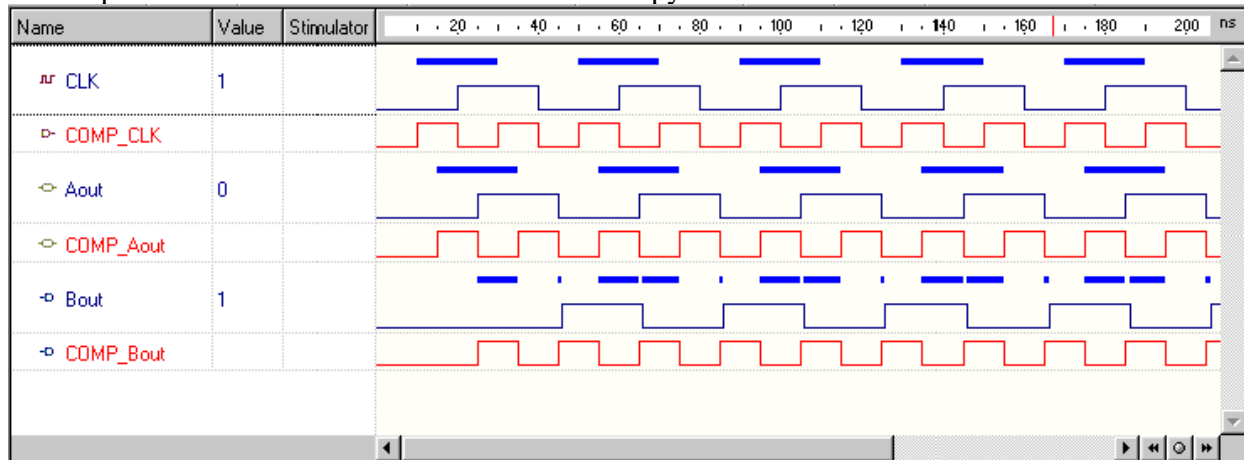
1. Створити новий проект в Active-HDL.
2. Створити об'єкт з наступним інтерфейсом:



- вхідний порт *CLK* типу **std_logic**,
- вхідний-вихідний порт *Ainout* типу **std_logic**;
- вихідний порт *Bout* типу **std_logic**.

3. В тіло архітектури включити наступний код:
4. **architecture** comp **of** comp **is**
5. **-- signal** CLK : **std_logic** := '0';
6. **begin**
7. Pr_CLK: **process**(CLK)
8. **begin**
9. **--** формування сигналу CLK
10. **end process** Pr_CLK;
11. Pr_A: **process**(CLK)
12. **begin**
13. **if** CLK'event and CLK = '1' **then** Ainout <= '1' **after** 5 ns;
14. **elsif** CLK'event and CLK = '0' **then** Ainout <= '0' **after** 5 ns;
15. **end if**;
16. **end process** Pr_A;
17. Pr_B: **process** (Ainout)
18. **begin**
19. **if** Ainout'event **then** Bout <= **not** Ainout;
20. **end if**;
21. **end process** Pr_B;
22. **end comp**;
23. Скопіювати створений об'єкт і промодельовати його роботу. Отримати часові діаграми для сигналів *CLK* (задається як джерело з синхронізуючим імпульсом частотою 50 МГц), *Ainout*, *Bout*.
24. Зберегти результати моделювання як файл з ім'ям Wform1.awf у біжучій директорії.
25. За допомогою оператора **wait for** сформувати синхронізуючий сигнал *CLK* самостійно (попередньо виключивши його з інтерфейсу пристрою) в тілі процесу *Pr_CLK*. Рівень цього сигналу повинен змінюватись кожні 10 ns.
26. Вивчити роботу процесу *Pr_A* та переписати його з використанням оператора **wait on**.
27. Вивчити роботу процесу *Pr_B* та переписати його з використанням оператора **wait until**.
28. Промодельовати роботу створених за допомогою оператора **wait** процесів, отримати часову діаграму.
29. Порівняти поточну часову діаграму та базову Wform1, для чого увійти до меню **Waveform \ Compare Waveforms** - на екрані з'явиться вікно "Open"; то вибрати

файл Wform1.awf та натиснути кнопку "Open" - в результаті до поточної часової діаграми будуть додані часові залежності, які зберігаються у Wform1.awf, темно-синім кольором позначені поточні часові залежності, червоним - додані, розбіжності між порівнюваними сигналами позначаються грубою блакитною лінією.



Якщо базова та біжуча діаграми співпадають, перейти до наступного пункту виконання роботи.

30. Вийти з режиму порівняння. Для цього необхідно вибрати у вікні часових діаграм будь-яку лінію розбіжності сигналів (грубу блакитну лінію), та натиснути праву кнопку миші, і у меню, що з'явилося, вибрати пункт "Remove Difference Marks".
31. Додати до послідовності операцій процесу *Pr_A* вираз **wait for 30 ns**.
32. Промодельовувати роботу процесу, записати отриману часову діаграму, як файл Wform2.awf.
33. У виразі, який був доданий у п.12, змінити час затримки на 20 ns.
34. Промодельовувати роботу процесу. Отриману часову діаграму порівняти з Wform2.awf.
35. Підготувати звіт до захисту.

Зміст звіту

1. Назва та мета виконання лабораторної роботи.
2. Основні види застосування оператора **wait**.
3. Варіант базового коду та відповідні результати моделювання (п. 4).
4. Варіанти коду із застосуванням оператора **wait** (пп. 6, 7, 8).
5. Результати порівняння (п. 9).
6. Результати моделювання (п. 15).
7. Висновки.